

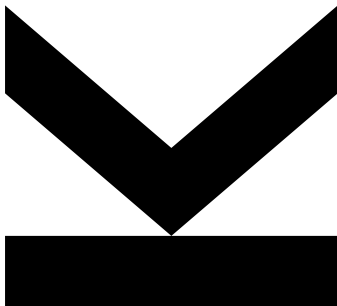
Sophie Öttl
k12308187,
Simon Grundner
k12136610,

Institut für Integrierte
Schaltungen

 <https://jku.at/iic>

5. Februar 2026

Protokoll



Digitale Schaltungstechnik PR - WS2025



JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Ampelsteuerung	5
1.1 Vorbereitung	5
2. Reaktionsspiel	5
2.1 Vorbereitung	5
2.1.1 Widerstandsberechnung	5
2.1.2 Fragen	5
2.2 Durchführung	5
3. Realisierung eines Logic Analyzers	6
3.1 Vorbereitung	6
3.2 Durchführung	6
3.3 Ergebnisse	6
4. Wertebestimmung von Widerstand und Kondensator	7
4.1 Vorbereitung	7
4.1.1 Widerstandsbestimmung	7
4.1.2 Spannungsteiler	7
4.1.3 Kondensatorladung	8
4.2 Durchführung	8
4.3 Ergebnisse	8
4.3.1 Widerstand 1	8
4.3.2 Widerstand 2	9
4.3.3 Widerstand 3	9
4.3.4 Widerstand 4	9
5. Ansteuerung eines Lautsprechers	10
5.1 Vorbereitung	10
5.1.1 2-Bit R2R Netzwerk	10
5.1.2 Sinus Wellentabelle	11
5.2 Durchführung	12
5.2.1 Aufbau am Steckbrett	12
5.2.2 Programmierung	12
5.3 Ergebnisse	12
6. Motorsteuerung mit Näherungssensor	13
6.1 Vorbereitung	13
6.1.1 Ultraschall-Modul HC-SR04 information	13
6.1.2 Plan für die Verwendung des Ultraschall Moduls	13
6.1.3 Schieberegister für 7 segment Anzeige	14
6.1.4 Berechnung Vorwiderstände	14
6.2 Praktikumsaufgabe	15
6.2.1 7 Segment	15
6.3 b	15
6.3.1 c	15
6.3.2 d	16
6.3.3 e	18

7. Datenübertragung mittels Infra-Rot (IR)	22
7.1 Vorbereitung	22
7.1.1 Morse Code Symbole	22
7.1.2 Codierung des Morsesignals	22
7.1.3 Widerstandsberechnung	23
7.2 Durchführung	23
7.2.1 Aufbau am Steckbrett	23
7.2.2 Implementierung des Senders	23
7.2.3 Implementierung des Empfängers	23
7.3 Ergebnisse	23

Geräteliste

Table 1: Caption

Pos.	Bezeichnung	Inventar Nr.
1	Multimeter	
2	Oszilloskop	

1. Ampelsteuerung

1.1 Vorbereitung

2. Reaktionsspiel

2.1 Vorbereitung

2.1.1 Widerstandsberechnung

Berechnet:

$$R_{V,calc} = \frac{5\text{ V} - 2\text{ V}}{15\text{ mA}} = 200\ \Omega$$

Aus E6-Reihe gewählt:

$$R_V = 220\ \Omega$$

2.1.2 Fragen

? Wie kann das Mikrocontroller-Programm zu einem beliebigen Zeitpunkt gestoppt werden?
Wie kann ausgewertet werden, ob der Taster zum richtigen oder zum falschen Zeitpunkt gedrückt wurde?

Das Programm kann mittels eines **externen Interrupts** zu einem beliebigen Zeitpunkt unterbrochen werden. Ein Interrupt wird dazu an einen Pin konfiguriert.

Sobald der Interrupt ausgelöst wird, springt das Programm direkt in die **Interrupt-Service-Routine (ISR)**. Dort werden Variablen gesetzt, welche die `loop`-Methode anschließend überprüft, um den Ablauf zu stoppen.

Zudem kann in der ISR der aktuelle Status (z.B. welche LED gerade aktiv ist) gespeichert werden. Da der Interrupt sofort reagiert, lässt sich so exakt auswerten, ob der Taster zum richtigen Zeitpunkt gedrückt wurde.

2.2 Durchführung

- Tinker CAD Link
- Stückliste

```
1 const int ledPins[] = {3, 4, 5, 6, 7};
2 const int buttonPin = 2;
3
4 for (int i = 0; i < 5; i++) {
5     pinMode(ledPins[i], OUTPUT);
6 }
7 pinMode(buttonPin, INPUT_PULLUP);
8 attachInterrupt(digitalPinToInterrupt(buttonPin), handlePress, FALLING);
```

Da nur die Pins 2 und 3 mit einem Interrupt konfiguriert werden können, wurde der Taster mit Pin 2 verbunden. Pin 3 bis 7 sind von den LEDs belegt.

3. Realisierung eines Logic Analyzers

3.1 Vorbereitung

AND		
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$$Y = A \wedge B$$

OR		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

$$Y = A \vee B$$

NAND		
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

$$Y = \overline{A \wedge B}$$

NOR		
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

$$Y = \overline{A \vee B}$$

XOR		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = A \oplus B$$

3.2 Durchführung

- Tinker CAD Link
- Stückliste

3.3 Ergebnisse

- Gatter 1: OR gate - Gatter 2: NAND gate - Gatter 3: XOR gate

Multimeter Serien Nr: IH807686

4. Wertebestimmung von Widerstand und Kondensator

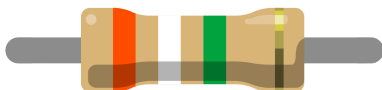
4.1 Vorbereitung

4.1.1 Widerstandsbestimmung



Widerstandswert: 47 kΩ

Toleranz: ±5 %



Widerstandswert: 3.9 MΩ

Toleranz: ±5 %



Widerstandswert: 6.8 kΩ

Toleranz: ±10 %

4.1.2 Spannungsteiler

- a) Die Spannung teilt sich am Spannungsteiler im Verhältnis der Widerstände auf.

$$U_{R1} = U_e - U_a = 5 \text{ V} - 3 \text{ V} = 2 \text{ V}$$

Das Verhältnis der Widerstände $R_1 : R_2$ entspricht dem Verhältnis der Spannungen $U_{R1} : U_a$, also **2:3**.

Berechnung von R_2 mit $R_1 = 10 \text{ kΩ}$:

$$\begin{aligned} \frac{R_1}{R_2} &= \frac{U_{R1}}{U_a} \\ R_2 &= R_1 \cdot \frac{U_a}{U_{R1}} \\ R_2 &= 10 \text{ kΩ} \cdot \frac{3 \text{ V}}{2 \text{ V}} \\ R_2 &= 15 \text{ kΩ} \end{aligned}$$

- b) Der Widerstand R_2 wird durch R_x ersetzt. Die neue Ausgangsspannung beträgt $U_a = 1 \text{ V}$.

$$U_{R1} = 5 \text{ V} - 1 \text{ V} = 4 \text{ V}$$

Berechnung von R_x :

$$\begin{aligned} R_x &= R_1 \cdot \frac{U_a}{U_{R1}} \\ R_x &= 10 \text{ kΩ} \cdot \frac{1 \text{ V}}{4 \text{ V}} \\ R_x &= 2.5 \text{ kΩ} \end{aligned}$$

4.1.3 Kondensatorladung

a) Die Spannung am Kondensator folgt der Ladefunktion:

$$u_c(t) = U_q \cdot \left(1 - e^{-\frac{t}{\tau}}\right)$$

Es ist bekannt, dass der Kondensator nach der Zeitkonstante $\tau = R \cdot C$ circa 63 % der Endspannung erreicht hat ($1 - e^{-1} \approx 0.63$).

Da nach $t = 100 \text{ ms}$ genau dieser Wert erreicht ist, gilt:

$$\tau = 100 \text{ ms}$$

Daraus lässt sich die Kapazität C berechnen:

$$\tau = R \cdot C$$

$$C = \frac{\tau}{R}$$

$$C = \frac{0.1 \text{ s}}{10 \text{ k}\Omega}$$

$$C = 0.00001 \text{ F} = 10 \text{ }\mu\text{F}$$

b) Der Kondensator gilt als vollständig geladen (Spannung $> 99\%$), wenn eine Zeit von ca. $5 \cdot \tau$ vergangen ist ($1 - e^{-5} \approx 0.993$).

Die Ladedauer beträgt somit:

$$t_{\text{voll}} \approx 5 \cdot \tau$$

$$t_{\text{voll}} = 5 \cdot 100 \text{ ms}$$

$$t_{\text{voll}} = 500 \text{ ms}$$

4.2 Durchführung

- Tinker CAD Link Aufgabe 1
- Tinker CAD Link Aufgabe 2
- Stückliste

4.3 Ergebnisse

4.3.1 Widerstand 1

Farbcode Braun Schwarz Schwarz Schwarz Braun

100 Ω

Multimeter gemessen: $100.3 \Omega \pm 1\%$

Programm hHalo

Table 2:

Widerstandswert / Ω	ADC Messwert
78.50	8
58.76	6
68.62	7
78.50	8

4.3.2 Widerstand 2

Multimeter gemessen: 12.01 k Ω

Farbcode

4.3.3 Widerstand 3

Multimeter gemessen: 185.7 k Ω

Farbcode Braun Graun Schwarz Orange Braun

180 k Ω $\pm$ 1 %

Programm HALlo

Table 3:

Widerstandswert / Ω	ADC Messwert
179090.73	970
186362.31	972
182657.92	971
194215.59	974

4.3.4 Widerstand 4

Multimeter gemessen: 1.01 k Ω

5. Ansteuerung eines Lautsprechers

5.1 Vorbereitung

5.1.1 2-Bit R2R Netzwerk

Ein 2-Bit Digital-Analog Wandler wird mit einem R2R-Netzwerk realisiert.

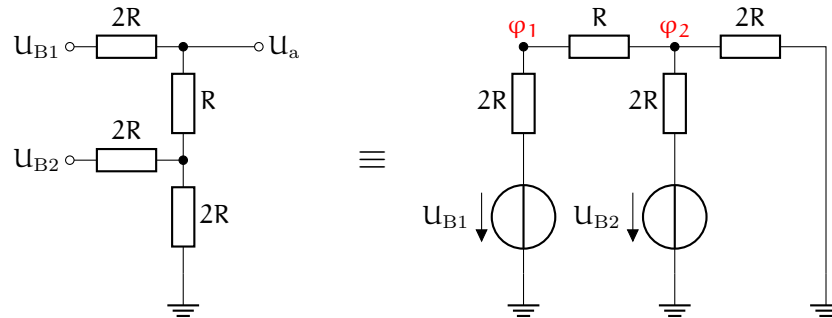


Figure 1: R2R Netzwerk

Um das Netzwerk zu lösen, wird das Knotenpotential-Verfahren verwendet. Das Netzwerk weist dabei zwei Knoten mit den Potenzialen φ_1 und φ_2 bezüglich Masse auf, wobei zu erkennen ist, dass $\varphi_1 = U_a$ ist. Es lässt sich ein Gleichungssystem der Form

$$\begin{bmatrix} G_{11} & -G_{12} \\ -G_{21} & G_{22} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} = \begin{bmatrix} I_1 \\ I_2 \end{bmatrix} \quad (1)$$

aufstellen. Dabei ist G_{ii} die Summe der an Knoten i anliegenden Leitwerte und G_{ij} die Summe der Leitwerte, welche Knoten i und Knoten j verbinden. Es gilt $G_{ij} = G_{ji}$ wenn $i \neq j$. I_i sind die zum Knoten i zufließenden Quellströme, diese können aus den realen Spannungsquellen mit der jeweiligen Ersatzstromquelle berechnet werden.

$$\begin{aligned} \begin{bmatrix} \frac{1}{2R} + \frac{1}{R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{1}{R} + \frac{1}{2R} + \frac{1}{2R} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \frac{1}{2R} \\ U_{B2} \frac{1}{2R} \end{bmatrix} \\ 2R \begin{bmatrix} \frac{3}{2R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{4}{2R} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \\ U_{B2} \end{bmatrix} \\ 2I + II \rightarrow \begin{bmatrix} 3 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \\ U_{B2} \end{bmatrix} \\ \begin{bmatrix} 4 & 0 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} 2U_{B1} + U_{B2} \\ U_{B2} \end{bmatrix} \end{aligned}$$

Aus der ersten Zeile des Gleichungssystems und mit $\varphi_1 = U_a$ folgt nun

$$4\varphi_1 = 2U_{B1} + U_{B2} \implies U_a = \frac{U_{B1}}{2} + \frac{U_{B2}}{4} \quad (2)$$

Die Ausgangswerte bei verschiedenen Eingangsstellungen sind in Tabelle 4 gezeigt. Das Muster in Gleichung 2 lässt sich auch für 4-Bit fortsetzen

$$U_{a,4\text{-Bit}} = \frac{U_{B1}}{2} + \frac{U_{B2}}{4} + \frac{U_{B3}}{8} + \frac{U_{B4}}{16}$$

Table 4: Ausgänge des 2-Bit R2R Netzwerkes bei verschiedenen Eingangsstellungen

U_{B1}	U_{B2}	U_a
0 V	0 V	0 V
0 V	U_{Bit}	$\frac{1}{4} U_{Bit}$
U_{Bit}	0 V	$\frac{1}{2} U_{Bit}$
U_{Bit}	U_{Bit}	$\frac{3}{4} U_{Bit}$

5.1.2 Sinus Wellentabelle

a) Die Amplitude eines Sinus wird über eine Periode in 20 Werte diskretisiert. Die diskreten Werte erhält man mit:

$$D_{SIN} = \text{round} \left(U_{SIN} \frac{D_{MAX}}{U_{MAX}} \right) = \text{round} \left(U_{SIN} \frac{15}{5 \text{ V}} \right)$$

Die Berechneten Werte sind in Tabelle 5 aufgeführt.

Table 5: Amplituden des Sinus im Bereich 0-5 V, unterteilt in 20 Stützwerte.

U_{SIN} / V	D_{SIN} Dezimal	D_{SIN} Hexadezimal
2.5	8	0x8
3.3	10	0xA
4.0	12	0xC
4.5	14	0xE
4.9	15	0xF
5.0	15	0xF
4.9	15	0xF
4.5	14	0xE
4.0	12	0xC
3.3	10	0xA
2.5	8	0x8
1.7	5	0x5
1.0	3	0x3
0.5	2	0x2
0.1	0	0x0
0.0	0	0x0
0.1	0	0x0
0.5	2	0x2
1.0	3	0x3
1.7	5	0x5

b) In einer Periodendauer T müssen alle $N = 20$ Werte der Wellentabelle durchlaufen werden. Die Zeit, wie lange ein Bitmuster am Ausgang anliegen muss, ergibt sich durch

$$t_b = \frac{T}{N} = \frac{1}{fN}.$$

Die berechneten Werte sind in Tabelle 6 dargestellt.

Table 6: Caption

Ton	Frequenz f / Hz	Zeit pro Bitmuster t_b / μs
c'	262	190.84
d'	294	170.07
e'	330	151.52
f'	349	143.27
g'	392	127.55
a'	440	113.64
h'	494	101.21

5.2 Durchführung

- Tinker CAD Link
- Stückliste

5.2.1 Aufbau am Steckbrett

5.2.2 Programmierung

5.3 Ergebnisse

6. Motorsteuerung mit Näherungssensor

- Tinker CAD Link
- Stückliste

6.1 Vorbereitung

6.1.1 Ultraschall-Modul HC-SR04 information

Folgende wichtige Spezifikationsdaten konnten auf Mikrocontroller.net gefunden werden:

- Messbereich: 2 cm bis ca. 300 cm.
- Auflösung: Systembedingt ca. 3 mm.
- Versorgung: 5V Spannung bei einer Stromaufnahme von weniger als 2 mA.
- Messrate: Maximal 50 Messungen pro Sekunde (Intervall von 20 ms).

Pin Belegung (Von links nach Rechts):

- **Pin 1:** VCC (5V)
- **Pin 2:** Trigger eingang TTL (0-0.8V ist low, 2,4-5V ist High)
- **Pin 3:** Ausgang Messergebnis TTL
- **Pin 4:** Ground

Eine Messung wird gestartet durch eine fallende Flanke am Triggereingang (Pin 2) die 10 µsec andauert.

Das modul sendet nach 250 µsec ein signal. Nun geht der ausgang auf High. Sobald das Echo empfangen wird geht der ausgang wieder auf low. Wird kein echo empfangen, bleibt der ausgang für 200ms auf High. Damit ergibt sich die entfernung als:

$$d = \frac{t \cdot 34,3 \text{ cm/ms}}{2}$$

Das Ergebnis kann durch Temperatur, rauhe oberflächen und schlechtem winkel verschlechtert werden.

6.1.2 Plan für die Verwendung des Ultraschall Moduls

Der Grobe Plan für die verwendung ist, dass der Trigger und der Ausgangs Pin mit dem Arduino verbunden werden. Es werden Interrupts für den ausgang konfiguriert. Bei steigender Flanke wird die aktuelle systemzeit gespeichert. Bei fallender Flanke wird mit der differenz der aktuellen zeit und der gespeicherten zeit die Distanz berechnet und ein boolean gesetzt um anzuzeigen, dass die messung bereit ist,

Im hauptprogramm wird der steuer pin kurz auf high und wieder auf low gesetzt um eine messung zu triggern. Anschließend wird ein delay von 30ms durchgeführt damit wir die maximale Messrate nicht unterschreiten. Anschließend wird falls der Mess-boolean gesetzt wurde der Motor Ausgeschaltet.

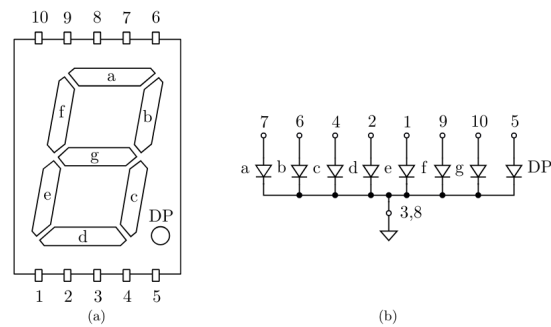


Figure 2: 7-Segment Anzeige

Ziffer	a	b	c	d	e	f	g	dp
0	1	1	1	1	1	1	0	0
1	0	1	1	0	0	0	0	0
2	1	1	0	1	1	0	1	0
3	1	1	1	1	0	0	1	0
4	0	1	1	0	0	1	1	0
5	1	0	1	1	0	1	1	0
6	1	0	1	1	1	1	1	0
7	1	1	1	0	0	0	0	0
8	1	1	1	1	1	1	1	0
9	1	1	1	1	0	1	1	0

Table 7: Segmentbelegung für 7-Segment-Anzeige (High-aktiv)

6.1.3 Schieberegister für 7 segment Anzeige

6.1.4 Berechnung Vorwiderstände

Die Benötigte DiodenSpannung der LEDs ist $U_D = 2V$ bei einem Strom $I_D = 15mA$. Es ist mit einer Spannung von $U_{CC} = 5V$ an den Ausgängen des Schieberegisters zu rechnen. Der Vorwiderstand Kann wie folgt berechnet werden:

$$U_{R_V} = U_{CC} - U_D = 5V - 2V = 3V$$

$$I_{R_V} = I_D = 15mA$$

$$R_V = \frac{U_{R_V}}{I_{R_V}} = \frac{3V}{15mA} = 200\Omega$$

Die E6 Reihe enthält die Werte (1,0 1,5 2,2 3,3 4,7 6,8). Damit sind die nächsten 2 Widerstände 150Ω und 220Ω . Da der Strom von $I_D = 15mA$ nicht überschritten werden soll, wählen wir den größeren Widerstand. Somit ist der Vorwiderstand: $R_V = 220\Omega$

6.2 Praktikumsaufgabe

6.2.1 7 Segment

6.3 b

6.3.1 c

```
1 // C++ code
2 //
3 const int SHIFT_IN=4;
4 const int SHIFT_OUTPUT=5;
5 const int SHIFT_CLK=6;
6
7 const int BUTTON_UP = 2;
8 const int BUTTON_DOWN = 3;
9
10 volatile int currentNumber = 0;
11 volatile bool needsUpdate = true;
12
13 unsigned long lastInterruptTime = 0;
14 const unsigned long debounceDelay = 200;
15
16 const uint8_t segmentPatterns[] = {
17     0b11111100, // 0
18     0b01100000, // 1
19     0b11011010, // 2
20     0b11110010, // 3
21     0b01100110, // 4
22     0b10110110, // 5
23     0b10111110, // 6
24     0b11100000, // 7
25     0b11111110, // 8
26     0b11110110 // 9
27 };
28
29 void setup()
30 {
31     pinMode(LED_BUILTIN, OUTPUT);
32
33     pinMode(SHIFT_IN, OUTPUT);
34     pinMode(SHIFT_CLK, OUTPUT);
35     pinMode(SHIFT_OUTPUT, OUTPUT);
36
37     pinMode(BUTTON_UP, INPUT_PULLUP);
38     pinMode(BUTTON_DOWN, INPUT_PULLUP);
39
40     attachInterrupt(digitalPinToInterrupt(BUTTON_UP), countUp, FALLING);
41     attachInterrupt(digitalPinToInterrupt(BUTTON_DOWN), countDown, FALLING);
42 }
43
44 void loop()
45 {
46     if(needsUpdate) {
47         needsUpdate = false;
```

```

48     setShiftRegister(currentNumber);
49 }
50 }
51
52 void countUp() {
53     unsigned long time = millis();
54     if (time - lastInterruptTime > debounceDelay) {
55         if(currentNumber >= 9) {
56             currentNumber = 0;
57         } else {
58             currentNumber++;
59         }
60         needsUpdate = true;
61     }
62     lastInterruptTime = time;
63 }
64
65 void countDown() {
66     unsigned long time = millis();
67     if (time - lastInterruptTime > debounceDelay) {
68         if(currentNumber <=0) {
69             currentNumber = 9;
70         } else {
71             currentNumber--;
72         }
73         needsUpdate = true;
74     }
75     lastInterruptTime = time;
76 }
77
78
79 void setShiftRegister(int number) {
80     digitalWrite(SHIFT_OUTPUT, LOW);
81
82     shiftOut(SHIFT_IN, SHIFT_CLK, LSBFIRST, segmentPatterns[number]);
83
84     digitalWrite(SHIFT_OUTPUT, HIGH);
85 }
86

```

6.3.2 d

```

1
2 // C++ code
3 //
4 const int SHIFT_IN=4;
5 const int SHIFT_OUTPUT=5;
6 const int SHIFT_CLK=6;
7
8 const int BUTTON_UP = 2;
9 const int BUTTON_DOWN = 3;
10
11 volatile int currentNumber = 0;
12 volatile bool needsUpdate = true;

```

```
13
14 unsigned long lastInterruptTime = 0;
15 const unsigned long debounceDelay = 200;
16
17
18 const int MOTOR_LED_OFF=9;
19 const int MOTOR_LED_ON=8;
20 const int MOTOR_PWM_LEFT=10;
21
22 volatile int motorSpeed = 0;
23
24 const uint8_t segmentPatterns[] = {
25     0b11111100, // 0
26     0b01100000, // 1
27     0b11011010, // 2
28     0b11110010, // 3
29     0b01100110, // 4
30     0b10110110, // 5
31     0b10111110, // 6
32     0b11100000, // 7
33     0b11111110, // 8
34     0b11110110 // 9
35 };
36
37 void setup()
38 {
39     pinMode(LED_BUILTIN, OUTPUT);
40
41     pinMode(SHIFT_IN, OUTPUT);
42     pinMode(SHIFT_CLK, OUTPUT);
43     pinMode(SHIFT_OUTPUT, OUTPUT);
44
45     pinMode(MOTOR_PWM_LEFT, OUTPUT);
46     pinMode(MOTOR_LED_OFF, OUTPUT);
47     pinMode(MOTOR_LED_ON, OUTPUT);
48
49     pinMode(BUTTON_UP, INPUT_PULLUP);
50     pinMode(MOTOR_LED_ON, OUTPUT);
51
52     attachInterrupt(digitalPinToInterrupt(BUTTON_UP), countUp, FALLING);
53     attachInterrupt(digitalPinToInterrupt(BUTTON_DOWN), countDown, FALLING);
54
55
56
57     setShiftRegister(currentNumber);
58     setMotorSpeed(currentNumber);
59 }
60
61 void loop()
62 {
63     if(needsUpdate) {
64         needsUpdate = false;
65         setShiftRegister(currentNumber);
66         setMotorSpeed(currentNumber);
67     }
68 }
69
```

```

70 void countUp() {
71     unsigned long time = millis();
72     if (time - lastInterruptTime > debounceDelay) {
73         if(currentNumber >= 9) {
74             currentNumber = 0;
75         } else {
76             currentNumber++;
77         }
78         needsUpdate = true;
79     }
80     lastInterruptTime = time;
81 }
82
83 void countDown() {
84     unsigned long time = millis();
85     if (time - lastInterruptTime > debounceDelay) {
86         if(currentNumber <=0) {
87             currentNumber = 9;
88         } else {
89             currentNumber--;
90         }
91         needsUpdate = true;
92     }
93     lastInterruptTime = time;
94 }
95
96
97 void setShiftRegister(int number) {
98     digitalWrite(SHIFT_OUTPUT, LOW);
99
100     shiftOut(SHIFT_IN, SHIFT_CLK, LSBFIRST, segmentPatterns[number]);
101
102     digitalWrite(SHIFT_OUTPUT, HIGH);
103 }
104
105 void setMotorSpeed(int number) {
106     int pwmValue = map(number, 0, 9, 0, 255);
107     analogWrite(MOTOR_PWM_LEFT, pwmValue);
108
109     if(number != 0) {
110         digitalWrite(MOTOR_LED_OFF, LOW);
111         digitalWrite(MOTOR_LED_ON, HIGH);
112     } else {
113         digitalWrite(MOTOR_LED_ON, LOW);
114         digitalWrite(MOTOR_LED_OFF, HIGH);
115     }
116 }

```

6.3.3 e

```

1 // C++ code
2 //
3 const int SHIFT_IN=4;
4 const int SHIFT_OUTPUT=5;

```

```
5  const int SHIFT_CLK=6;
6
7  const int BUTTON_UP = 2;
8  const int BUTTON_DOWN = 3;
9
10 volatile int currentNumber = 0;
11 volatile bool needsUpdate = true;
12
13 unsigned long lastInterruptTime = 0;
14 const unsigned long debounceDelay = 200;
15
16
17 const int MOTOR_LED_OFF=9;
18 const int MOTOR_LED_ON=8;
19 const int MOTOR_PWM_LEFT=10;
20
21 volatile int motorSpeed = 0;
22
23 const int SENSOR_TRIGGER = 13;
24 const int SENSOR_ECHO = 12;
25
26 unsigned long lastSensorTime = 0;
27 unsigned long sensorPeriodMillis = 60;
28
29
30 const uint8_t segmentPatterns[] = {
31     0b11111100, // 0
32     0b01100000, // 1
33     0b11011010, // 2
34     0b11110010, // 3
35     0b01100110, // 4
36     0b10110110, // 5
37     0b10111110, // 6
38     0b11100000, // 7
39     0b11111110, // 8
40     0b11110110 // 9
41 };
42
43 void setup()
44 {
45     Serial.begin(9600);
46
47     pinMode(LED_BUILTIN, OUTPUT);
48
49     pinMode(SHIFT_IN, OUTPUT);
50     pinMode(SHIFT_CLK, OUTPUT);
51     pinMode(SHIFT_OUTPUT, OUTPUT);
52
53     pinMode(MOTOR_PWM_LEFT, OUTPUT);
54     pinMode(MOTOR_LED_OFF, OUTPUT);
55     pinMode(MOTOR_PWM_LEFT, OUTPUT);
56
57     pinMode(MOTOR_LED_OFF, OUTPUT);
58     pinMode(MOTOR_PWM_LEFT, OUTPUT);
59
60     pinMode(BUTTON_UP, INPUT_PULLUP);
61     pinMode(MOTOR_LED_ON, OUTPUT);
```

```
62
63   attachInterrupt(digitalPinToInterrupt(BUTTON_UP), countUp, FALLING);
64   attachInterrupt(digitalPinToInterrupt(BUTTON_DOWN), countDown, FALLING);
65
66   pinMode(SENSOR_TRIGGER, OUTPUT);
67   pinMode(SENSOR_ECHO, INPUT);
68
69   setShiftRegister(currentNumber);
70   setMotorSpeed(currentNumber);
71   digitalWrite(SENSOR_TRIGGER, HIGH);
72 }
73
74 void loop()
75 {
76   if(needsUpdate) {
77     needsUpdate = false;
78     setShiftRegister(currentNumber);
79     setMotorSpeed(currentNumber);
80   }
81
82   unsigned long time = millis();
83   if(time - sensorPeriodMillis > lastSensorTime) {
84     float distance = getDistance();
85     Serial.print(distance);
86     Serial.println(" cm");
87     lastSensorTime = time;
88   }
89 }
90
91 void countUp() {
92   unsigned long time = millis();
93   if (time - lastInterruptTime > debounceDelay) {
94     if(currentNumber >= 9) {
95       currentNumber = 0 ;
96     } else {
97       currentNumber++;
98     }
99     needsUpdate = true;
100   }
101   lastInterruptTime = time;
102 }
103
104 void countDown() {
105   unsigned long time = millis();
106   if (time - lastInterruptTime > debounceDelay) {
107     if(currentNumber <=0) {
108       currentNumber = 9;
109     } else {
110       currentNumber--;
111     }
112     needsUpdate = true;
113   }
114   lastInterruptTime = time;
115 }
116
117
118 void setShiftRegister(int number) {
```

```
119     digitalWrite(SHIFT_OUTPUT, LOW);
120
121     shiftOut(SHIFT_IN, SHIFT_CLK, LSBFIRST, segmentPatterns[number]);
122
123     digitalWrite(SHIFT_OUTPUT, HIGH);
124 }
125
126 void setMotorSpeed(int number) {
127     int pwmValue = map(number, 0, 9, 0, 255);
128     analogWrite(MOTOR_PWM_LEFT, pwmValue);
129
130     if(number != 0) {
131         digitalWrite(MOTOR_LED_OFF, LOW);
132         digitalWrite(MOTOR_LED_ON, HIGH);
133     } else {
134         digitalWrite(MOTOR_LED_ON, LOW);
135         digitalWrite(MOTOR_LED_OFF, HIGH);
136     }
137 }
138
139 float getDistance() {
140     digitalWrite(SENSOR_TRIGGER, LOW);
141     delayMicroseconds(20);
142
143     long duration = pulseIn(SENSOR_ECHO, HIGH, 200000);
144     if (duration <= 0) duration = 200000;
145     float distance = (duration * 0.0343) / 2;
146
147     digitalWrite(SENSOR_TRIGGER, HIGH);
148     return distance;
149 }
```

7. Datenübertragung mittels Infra-Rot (IR)

Mit zwei Arduino Uno Boards soll eine Übertragung von Morsecodes mittels Infra-Rot Lichtsignalen aufgebaut werden. Sender seitig wird eine IR-LED angesteuert, sodass Infrarotsignale mit einer Frequenz von 38 kHz gesendet werden. Die Symbole der Morsezeichen (–, •) werden dabei mit unterschiedlicher Sendedauer enkodiert. Als Empfänger wird das IR-Detektor-Modul KY-022 verwendet, welches am Ausgang LOW ausgibt, wenn es ein IR-Lichtsignal mit der genannten Frequenz detektiert, ansonsten HIGH.

7.1 Vorbereitung

7.1.1 Morse Code Symbole

Eine Liste der Morse-Codewörter für das Alphabet ist in Tabelle 8 aufgeführt.

Table 8: Morsealphabet

Buchstabe	Code	Buchstabe	Code	Buchstabe	Code
A	• –	J	• – – –	S	• • •
B	– • • •	K	– • –	T	–
C	– • – •	L	• – • •	U	• • –
D	– • •	M	– –	V	• • • –
E	•	N	– •	W	• – –
F	• • – •	O	– – –	X	– • • –
G	– – •	P	• – – •	Y	– • – –
H	• • • •	Q	– – • –	Z	– – • •
I	• •	R	• – •		

7.1.2 Codierung des Morsesignals

Um die Symbole der Morse-Codewörter zu übertragen, wird mit der Funktion `tone(pin, frequency, duration)` ein 38 kHz Rechtecksignal mit unterschiedlicher Dauer auf dem LED-GPIO Pin ausgegeben. Damit die Methode `tone()` funktioniert, muss der Pin ein PWM Pin sein, z.B. 3 oder 11. Um die richtigen Symbol Dauern für die Symbole in einem Morse-Codewort zu senden, kann ein Ablauf wie in Algorithmus 1 verwendet werden.

Algorithmus 1 Ablauf zum Senden der Korrekten Symbol dauern in einem Morse-Codewort

```

for all Symbols in Codeword do
  if Symbol is Dash then
    tone(TX_PIN, F_IR_HZ, DASH_TIME_MS)
  else
    tone(TX_PIN, F_IR_HZ, DOT_TIME_MS)
  end if
  delay(INTER_SYMBOL_TIME_MS)
end for
delay(PAUSE_TIME_MS)

```

7.1.3 Widerstandsberechnung

Für die IR-Diode soll sich im Betrieb ein Arbeitspunkt von $I = 130 \text{ mA}$ und $U_F = 1.6 \text{ V}$ einstellen. Um den Strom durch die Diode einzustellen, muss der Widerstand R_2 passend dimensioniert werden.

Die restliche Spannung die nicht über die Diode abfällt, muss über R_2 und $R_{DS,on}$ abfallen. Mit $R_{DS,on} = 14 \Omega$ ergibt sich:

$$R_2 = \frac{5 \text{ V} - 1.6 \text{ V}}{130 \text{ mA}} - R_{DS,on} = 12.15 \Omega$$

Um den Maximalstrom nicht zu überschreiten, wird der nächst höhere Widerstand aus der E6 Reihe gewählt.

$$R_2 = 15 \Omega$$

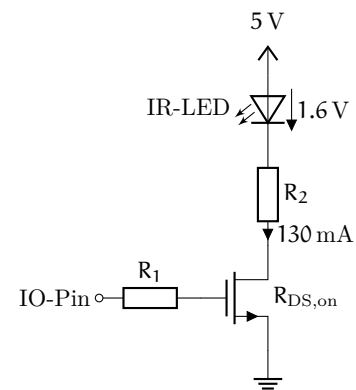


Figure 3: NMOS Treiber für die Infrarot LED.

7.2 Durchführung

- Tinker CAD Link
- Stückliste

7.2.1 Aufbau am Steckbrett

7.2.2 Implementierung des Senders

7.2.3 Implementierung des Empfängers

7.3 Ergebnisse